

Rapport Projet Fin de Semestre

Description générale du projet réalisé

Notre projet consiste en un réveil-matin de voyage permettant d'afficher et modifier l'heure courante, régler une alarme sonore, déclencher un chronomètre et finalement, régler et déclencher un compteur à rebours. Il marche à 1 MHz.



Mode d'emploi

Au démarrage, effectuer un reset global en actionnant le switch **Global Reset**¹. L'appareil se trouve par défaut en mode **CLOCK**, qui affiche l'heure courante. Les deux écrans 7-segments tout à gauche indiquent par deux lettres le mode dans lequel se trouve l'appareil.

- L'appui du bouton **M** = "**Mode**" permet passer de mode en mode de façon cyclique, selon le tableau présenté ci-dessous.
- L'appui du bouton **A** = "**Action**" effectue l'action propre au mode courant, détaillé dans le tableau ci-dessous.

N°	Mode Courant	Action
1	CLOCK	Rien.
2	CLOCK SET HOURS	Incrémente les h.
3	CLOCK SET MINUTES	Incrémente les min.
4	ALARM SET HOURS	Incrémente les h.
5	ALARM SET MINUTES	Incrémente les min.
6	TIMER IDLE	Lance le chrono.

N°	Mode Courant	Action
7	TIMER RUN	Fige le chrono.
8	CD SET MINUTES	Incrémente les min.
9	CD SET SECONDS	Incrémente les sec.
10	CD IDLE	Lance le c-à-r.
11	CD RUN	Fige le c-à-r.
12	CD END	Passe en mode 8.
13	ALARM REACHED	Passe en mode 4.

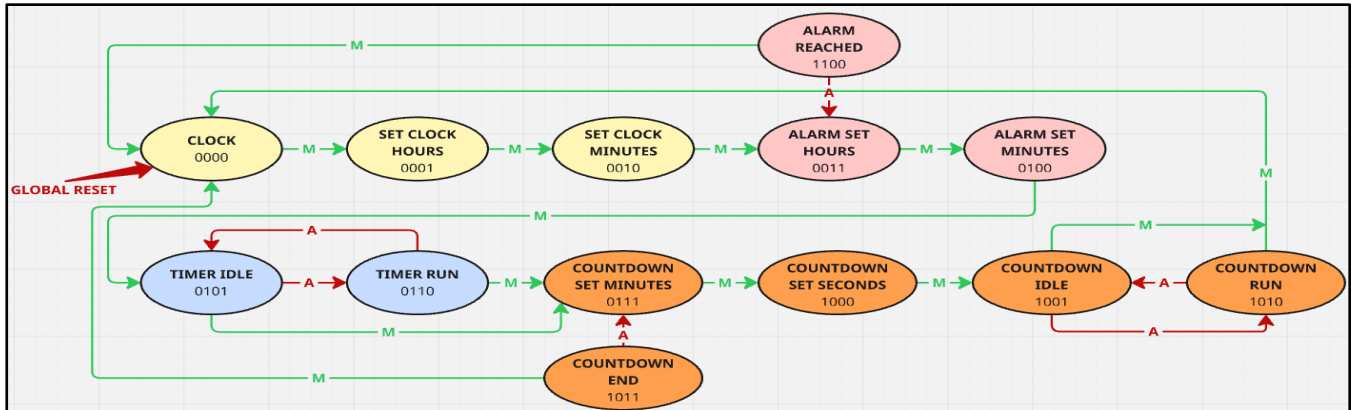
- Dans les modes **SET**, la valeur modifiée est celle qui clignote. Le dip-switch **Incr. direction switch** permet de régler la direction d'incrémentatation: pos. basse = incrémentatation, pos. haute = décrémentation
- Le switch **Enable alarm**, lorsqu'en position haute, active l'alarme, c-à-d. déclenche un signal sonore lorsque l'heure courante est égale à l'heure d'alarme réglée par l'utilisateur. Lorsque l'appareil se trouve en mode **ALARM REACHED**, basculer **Enable alarm** vers le bas² pour ensuite pouvoir utiliser les inputs dédiés à la sortie du mode (voir FSM générale).
- Le switch **Local timer reset** permet de remettre le chronomètre à 0.
- Fonctionnalité spéciale: lorsqu'en mode **CD END** ou **ALARM REACHED**, l'utilisateur a l'option d'éteindre le signal sonore et de repasser en mode **CLOCK** en secouant l'appareil. Pour permettre ceci, basculer le switch **Enable accel** à la position haute.

¹ Faire un aller-retour.

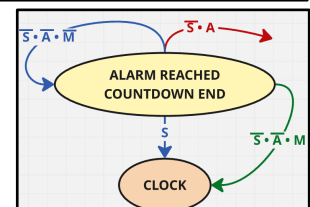
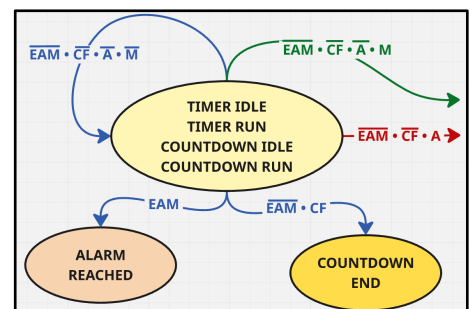
² Sinon, vous pourrez pas sortir du mode de déclenchement

Description de la FSM générale

Notre FSM générale comporte **13 modes**, ce qui peut sembler élevé pour un appareil à 4 fonctions principales. Ceci découle du principe guideur suivant : dans chaque état, le bouton Action ne doit exécuter qu'une seule et unique fonction. C'est donc la FSM qui détermine précisément, selon le mode attribué, l'unique fonction associée au bouton



Action, par des signaux de contrôle. Le diagramme de transitions général apparaît ci-dessus. Ce dernier indique uniquement les transitions dues aux deux boutons poussoirs et ne prend en compte ni les autres signaux en entrée de la FSM générale, ni les concepts de complétude et de cohérence. En effet, en plus des boutons poussoirs et l'état courant, la FSM générale reçoit les signaux: **EAM = Enabled Alarm Match**, **CF = Countdown Finished** et **S = Shake**. EAM et CF sont des signaux provenant des modules pilotés par la FSM générale indiquant si l'alarme réglée par l'utilisateur a été activée et est égale à l'heure courante, et si un compteur à rebours activé par l'utilisateur atteint 0, respectivement. Dans les deux cas, quel que soit le mode courant, la FSM ramène l'état courant à **ALARM REACHED** ou **COUNTDOWN END**, comme illustré dans le diagramme de transition détaillée des modes **IDLE/RUN**³. Celui-ci respecte les règles de complétude et de cohérence, qui ont nécessité un système de priorité entre les différents signaux (c.f. tableau). Finalement, la FSM possède un Reset Global qui ramène l'état au mode **CLOCK** de façon asynchrone.



Solutions à l'utilisation des périphériques G-sensor et Buzzer

Comme mentionné précédemment, secouer l'appareil permet de sortir des modes où le Buzzer est activé (**ALARM REACHED** et **COUNTDOWN END**). Ceci se fait par l'utilisation du driver G-sensor mis à disposition, **accel_final_driver**, qui indique en sortie si le sensor détecte une force élevée selon son axe vertical ou horizontal. Cette information est transférée par les sorties **x_large** et **y_large**, que nous traitons par une porte OR pour ainsi générer le signal **S = "Shake"** en entrée de la FSM.

Signal	Prior.
EAM	1
CF	2
S	3
A	4
M	5

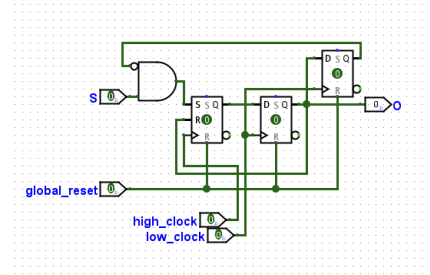
Fonctionnalités complexes: modules SR-Sync et SR-Pulse

Pour le traitement d'entrées des boutons **Action** et **Mode**, "debouncing" inclus, nous avons implémenté deux modules. Ces modules servent à échantillonner les états des boutons à une fréquence de **64Hz** (mise sur l'entrée "**high_clock**"), puis de synchroniser une éventuelle détection d'entrée sur **S** à l'horloge interne du système, qui opère à une fréquence de **16Hz** (mis sur l'entrée "**low_clock**").

64Hz a été choisi pour ne laisser entrer que des appuis plus longs de 15ms, et pour filtrer en même temps les "bounces" des boutons. Les deux modules traitent le signal de manière différente après son synchronisation:

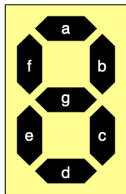
³ Le diagramme de transition détaillé des modes restants a été omis par manque d'espace, mais le principe reste le même.

- **SR-Sync:** si l'entrée est maintenue, la sortie du module sera un clock à 4Hz
 - Application: **Action** - l'incréméntation du temps dans un mode **SET** peut être fait en maintenant le bouton **A**
- **SR-Pulse:** pour n'importe-quelle entrée plus longue que 15ms, la sortie sera une seule pulsation synchronisée
 - Application: **Mode** - un changement de mode ne va jamais sauter plusieurs modes pendant un appui



Voici à droite notre implémentation de **SR-Sync**. En grandes lignes, le SR-latch sert à échantillonner l'entrée **S = Signal/Set** à 64Hz et la stocker. Les DFF en cascade après le SR-latch, mis à 16Hz, servent à envoyer un signal au reset du SR-latch, et, un tick plus tard (dans le cas de maintien du bouton), "renouveler" le signal à l'entrée du SR-latch pour finir un cycle de clock à 4Hz. **SR-Pulse** opère avec une idée similaire, mais avec un détecteur de rising edge utilisé à la fin. Malgré le fait que **SR-Pulse** marche en théorie et sur le simulateur, il paraît qu'il y a un bug lié à la métastabilité des DFFs dans l'implémentation sur la carte.

Problèmes résolues avec solution originale



Display

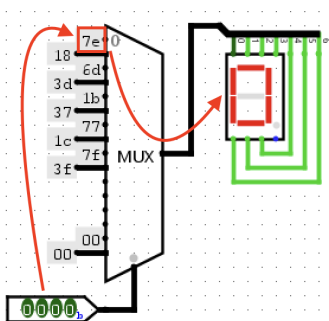
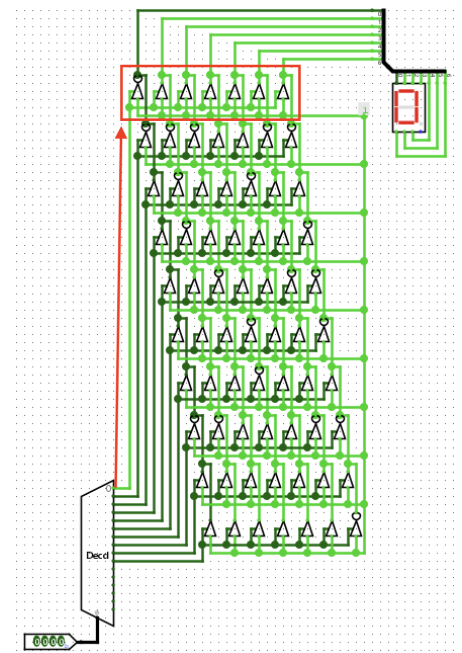
Notre première solution pour l'affichage se basait sur un **Demultiplexer**, avec l'idée principale d'associer une sortie du Demux avec 7 bits qui correspondent à un chiffre sur un 7-segment display. Notre premier système se basait sur des **buffers** et des **NOT-buffers**, qui étaient tous branchés sur une valeur constante à logique 1 et contrôlés par la sortie du Demux.

Dans ce système, chaque ligne correspond à un encodage de 7 bits associés à une sortie de la Demux. Ces 7 bits correspondent à un chiffre selon l'ordre suivant:

g - f - a - b - c - d - e

Voici à droite le premier circuit, avec l'exemple de 0 affiché. **0** correspond à l'encodage **0111111**, la ligne de buffers qui est activée par l'entrée **0000** dans le Demux. Ce système a pour avantage d'être flexible dans l'encodage, c'est à dire c'est un système qui permet l'encodage des lettres (ce qui n'est pas possible avec le module "BCD to 7-segment" de logisim).

Par contre, ce système n'était pas accepté sur la carte DE10-Lite, et il fallait donc le changer.



Voici à gauche le système changé pour être adapté à la carte. Il se base sur un **Multiplexer** au lieu d'un **Demux**, mais l'idée principale reste la même. Ici, nous encodons avec des "constantes à 7 bits," ce que logisim représente avec un numéro hexadécimal. Cette adaptation fait inverser la correspondance des LEDS, donc dans l'ordre suivant:

e - d - c - b - a - f - g

L'exemple donné, **0**, correspond à l'encodage **1111110**, égale à **0x7e**, et est activée par l'entrée **0000**. Ce nouveau système permet toujours la flexibilité du premier, mais est beaucoup plus simple et efficace. Avec ce système, nous obtenons le module **display**, avec 4 Mux associés à l'affichage de chaque chiffre et 2 Mux associés à l'affichage des lettres. Il y a aussi des Mux intermédiaires qui servent à contrôler les 6 Mux principales pour effectuer des "flashes," en cas de **CD END** ou **ALARM REACHED**, ou dans un mode **SET**. Pour créer les flashes, on a utilisé un gated clock mais qui ne pose aucun problème, vu qu'il opère à une fréquence très basse.